

5.1.2 Control Flow Graphs

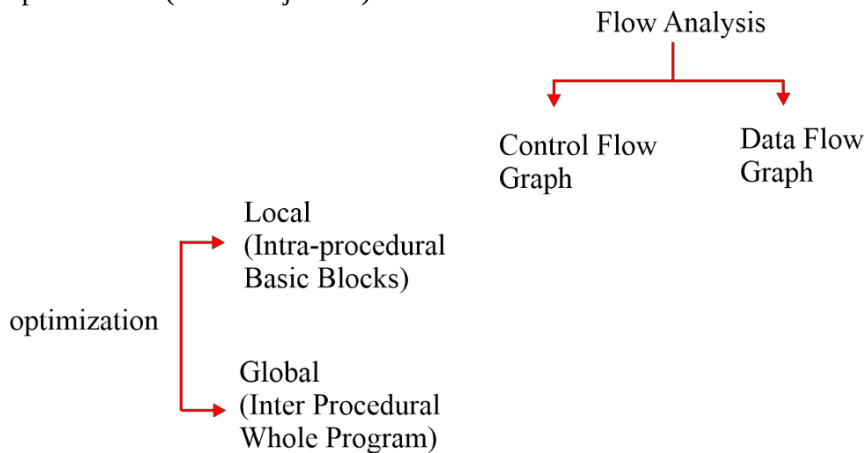
- CFG contain group of basic blocks and controls. CFG has nodes and edges to define basic blocks and controls.
- Basic Blocks: Sequence of 3-address code statements, in which control enters only from 1st statement (called as leader), and leaves from last statement.

Example:

1. $i = 1$	}	LB ₁
2. $j = 1$		LB ₂
3. $t1 = 5 * i$	}	LB ₃
4. $t2 = t1 + 5$		
5. $t3 = 4 * t2$		
6. $t4 = t3$		
7. $a[t4] = 1$	}	LB ₄
8. $j = j + 1$		
9. $\text{if } j \leq 5 \text{ goto } 3$	}	LB ₅
10. $i = i + 1$		
11. $\text{if } i < 5 \text{ goto } 2$	}	LB ₆

5.2 Code Optimization

Saves space / time. (Basic Objective)



5.2.1 Optimization Methods

- Constant folding
- Copy propagation
- Strength Reduction
- Dead code elimination
- Common sub expression elimination.
- Loop Optimization
- Peephole Optimization

1. Constant Folding

(i) $x = 2 * 3 + y \Rightarrow x = 6 + y$

Folding

ii) $x = 2 + y * 3$ can't fold the constant

2. Copy Propagation

i) Variable Propagation:

$x = y;$

$z = y + 2;$

$\Rightarrow z = x + 2;$

ii) Constant Propagation:

$x = 3$

$z = 3 + a;$

$\Rightarrow z = x + a$

3. Strength Reduction:

Replace expensive statement / instruction with cheaper one.

(i) $x = 2 * y$ **costly** $\Rightarrow x = y + y$; **Cheap**

(ii) $x = 2 * y \Rightarrow x = y < 1$; **Much Cheaper**

(iii) $x = y / 8 \Rightarrow x = y > 3;$

4. Dead Code Elimination:

$x = 2$; **FALSE**

if ($x > 2$)

printf ("code"); **DEAD CODE CAN BE REMOVED**

else

printf ("optimization");

$x = 2;$
printf ("optimization");

- Hence, above dead code never executes during execution. We can always delete such code.

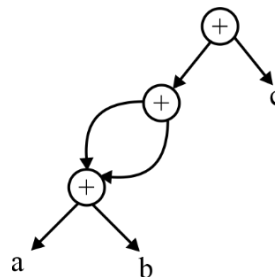
5. Common Sub Expression Elimination:

DAG is used to eliminate common sub expression.

Example: $x = (a + b) + (a + b) + c;$

$\Rightarrow t_1 = a + b$

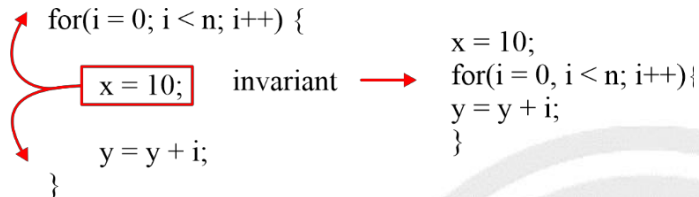
$x = t_1 + t_1 + c$



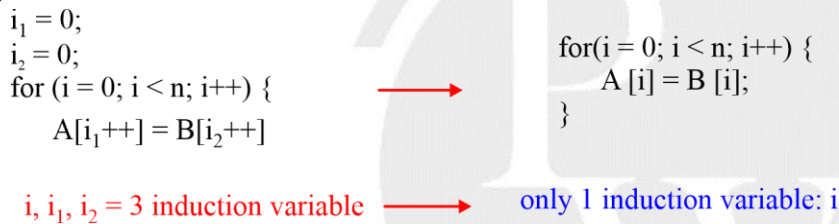
6. Loop Optimization:

(i) Code Motion – Frequency Reduction:

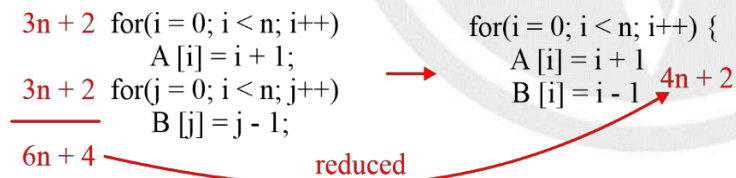
Move the loop invariant code outside of loop.



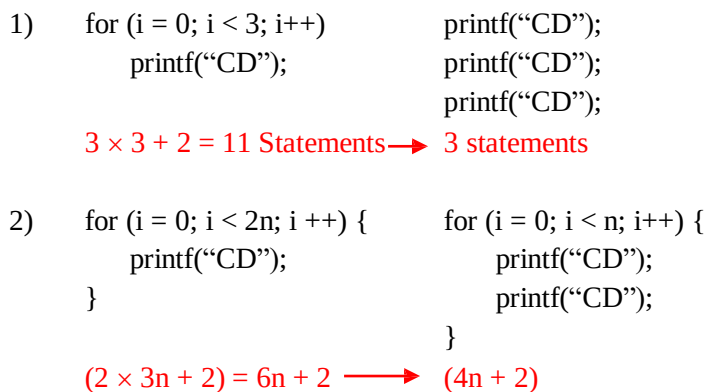
(ii) Induction Variable elimination:



(iii) Loop Merging / Combining: (Loop Jamming)



(iv) Loop Unrolling:



7. Peephole Optimization:

Examines a short sequence of target instructions in a window (*peephole*) and replaces the instructions by a faster and/or shorter sequence when possible.

- Applied to intermediate code or target code
- Following Optimizations can be used:



- Redundant instruction elimination
- Flow-of-control optimizations
- Algebraic simplifications
- Use of machine idioms

